PROYECTO DE CÁTEDRA · AUTÓMATAS Y COMPILADORES

ESPECIFICACIÓN DE INGENIERÍA · VERSIÓN 2.0

PSEUDO / STUDIO

Requerimientos completos para implementación operativa

Analizar · traducir · ejecutar · verificar

ENTREGA TÉCNICA

Funcionalidad real de extremo a extremo

ALCANCE INTEGRAL

29 requisitos funcionales
12 no funcionales

COBERTURA OFICIAL

Programas A–E
30 ejecuciones reales

DOCUMENTO REVISADO · 10 OCTUBRE 2026

VALIDACIÓN ACADÉMICA Y DE INGENIERÍA

ÍNDICE DEL DOCUMENTO

Guía de lectura

Una única especificación, sin módulos simulados ni funcionalidades relegadas a versiones futuras. El PDF detalla el contrato que el agente constructor debe materializar en código y pruebas.

1. Propósito, contexto y frontera académica
2. Usuarios, escenarios y experiencia general
3. Requerimientos funcionales obligatorios
4. Especificación formal del lenguaje fuente
5. Arquitectura integral e interconexiones
6. Generación de código, equivalencias y exportación
7. Casos académicos obligatorios: A–E
8. Matriz transversal de verificación
9. Requisitos no funcionales y operación (RNF)
10. Diseño de persistencia, estado y sincronización
11. Observabilidad funcional y modo defensa
12. Entregables físicos exigidos al agente constructor
13. Definición de terminado y protocolo de verificación
14. Incertidumbres preservadas de los documentos académicos
15. Correspondencia con documentos originales y control de versión

El archivo **PLAN_IMPLEMENTACION_AGENT.md** transforma cada requisito en tareas verificables y dependencias técnicas.

Especificación de requerimientos v2.0 — Producto integral, ejecutable y verificable
Proyecto de cátedra: Autómatas y Compiladores | 10 de octubre de 2026
Estado: documento prescriptivo para implementación por un agente de desarrollo; **no representa software ya implementado**.

Mandato de implementación. Construir un producto realmente operativo de extremo a extremo. No entregar pantallas decorativas, resultados prefijados, respuestas simuladas, árboles inventados, generación de código de ejemplo ni botones sin acción. Los módulos RF-01 a RF-18 y los nuevos RF-19 a RF-29 forman un alcance único obligatorio. Un requisito se cierra solamente cuando pasa pruebas automatizadas e integración real en interfaz, API, motor y ejecutores cuando corresponda. No crear categorías «MVP», «V2», «mock», «solo demostración» ni exclusiones por fase de desarrollo.

1. Propósito, contexto y frontera académica

Pseudo / Studio es una aplicación web local para escribir y abrir pseudocódigo, analizarlo con técnicas de compiladores, convertirlo **de verdad** a programas autónomos Python 3 y JavaScript (Node.js), ejecutarlos con entradas reales en un entorno aislado y visualizar de forma rastreable cada fase, los errores y la salida en consola. El usuario puede crear, guardar, recuperar, exportar, probar y explicar proyectos sin salir de la aplicación.

El archivo de asignación `fase_1.pdf` distingue **fase 1 académica** (investigación, arquitectura, BNF/EBNF y traducciones manuales A/B, entrega indicada el 11 de octubre) del **funcionamiento del compilador completo** a evaluar con los programas A–E y tres pruebas por programa. Esta especificación introduce un **objetivo de ingeniería adicional elegido por el equipo**: construir ahora todo el producto, aunque ello exceda la entrega documental de la primera fase. No afirmar que la consigna académica exige una web ejecutable en esa primera entrega.

Fuentes: requerimiento original `Requerimientos_Traductor_Pseudocodigo_Fase1.pdf` (v1.0), `fase_1.pdf` (6 páginas, incluidos programas A–E y rúbrica), `UIA2.pdf` (autómatas finitos) y `UIA3.pdf` (expresiones regulares y gramáticas). La gramática extendida, la API y las decisiones de seguridad de este documento son **especificación de ingeniería propuesta**, no citas literales del docente.

1.1 Resultado integral obligatorio

1. Editor interactivo con análisis real de fuente y vista sincronizada de tokens, árbol de sintaxis abstracta (AST), tabla de símbolos, diagnósticos y trazabilidad de fases.
2. Conversión determinista de una misma fuente a **dos** archivos ejecutables independientes (.py y .js) que preserven semántica y formato de salida.
3. Ejecución auténtica de ambos destinos con datos suministrados por el usuario; soporte tanto de ejecuciones por lotes como de consola interactiva (solicitudes `Leer` durante ejecución).
4. Cobertura plena de expresiones, entrada/salida, condicionales anidados, bucles `Para`, arreglos con `Dimension`, funciones con parámetros y retorno, y las extensiones de lenguaje definidas en esta especificación.
5. Gestión íntegra de proyectos, archivos, ejemplos, historial local de trabajo, descarga de fuentes/artefactos/informes y pruebas con evidencias reproducibles.
6. Aislamiento robusto para código generado, manejo seguro de fallos, accesibilidad y soporte verificable en navegadores modernos.

1.2 Reglas de cierre no negociables

- **Una función visual debe producir una operación real.** Todos los botones, pestañas, estados, resultados y mensajes provienen del backend o del estado real de la aplicación.
- **El análisis es previo a generar.** Cualquier error léxico, sintáctico o semántico bloquea producción y ejecución, pero no la consulta de diagnósticos.
- **Ejecución y traducción son fases diferentes.** Un código válido puede fallar al ejecutarse con ciertos datos; informar la clase y el punto del fallo sin presentarlo como error de compilación.
- **Equivalencia real.** Para una misma fuente y entradas, Python y Node.js deben producir salida observable equivalente; las divergencias no se ocultan ni se fuerzan mediante resultados prefijados.
- **No enseñar resultados inventados.** La interfaz distingue con texto y metadatos los datos de ejemplo de una ejecución real; las capturas documentales nunca constituyen evidencia técnica suficiente.
- **No prometer seguridad absoluta.** La ejecución se habilita solo después de comprobar que existe aislamiento operativo y controles efectivos; ante fallo de aislamiento mostrar error explícito, no sustituirlo por un simulador.
- **No restringirlo a A/B, A–E o cadenas codificadas.** Estas son pruebas de referencia; el compilador debe procesar cualquier fuente válida según la gramática pública, incluidos casos inéditos.

2. Usuarios, escenarios y experiencia general

Roles funcionales: estudiante/autora o autor de pseudocódigo; evaluador/docente que explora y comprueba etapas; operador local que configura la seguridad de los ejecutores. La instalación local individual no requiere cuentas de usuario ni depender de servicios SaaS. Si se implementa acceso remoto, se añaden autenticación, TLS, políticas CORS y límites por usuario **antes de habilitarlo**, nunca se expone el ejecutor públicamente por defecto.

Flujo principal: crear/abrir proyecto → editar fuente → analizar → inspeccionar tokens/AST/símbolos/errores → generar ambos destinos → comparar/differences → ejecutar Python o JavaScript con datos reales → revisar stdout/stderr, exit code y tiempo → guardar/exportar evidencias. El usuario puede regresar a editar, volver a analizar y descartar resultados obsoletos; todo artefacto tiene un hash de fuente y versión de compilador.

Interfaz esperada: escritorio web responsivo, diseño profesional similar en identidad visual al documento anterior (azul marino, acentos turquesa, paneles claros), editor con numeración, resaltado, selección y atajos; inspector colapsable; comparador de dos columnas; consola con scroll, envío de entrada y botones ejecutar/detener; panel de pruebas; ayuda sobre gramática; modo de defensa académica **alimentado por llamadas reales**, no por datos estáticos. Estados consistentes «pendiente / procesando / correcto / falló / cancelado / no disponible» con etiquetas textuales y nunca basados solo en color.

3. Requerimientos funcionales obligatorios

ID	Funcionalidad real y criterio de aceptación resumido
RF-01	Editor de pseudocódigo con resaltado, líneas, indentación, deshacer/rehacer, atajos, cargar archivos y ejemplos; fuente editable se usa en el análisis.
RF-02	Selector Python/JavaScript y vista simultánea; genera artefactos distintos procedentes del mismo AST y permite descargar ambos.

ID	Funcionalidad real y criterio de aceptación resumido
RF-03	Lexer propio: tokens reales con clase, lexema, offset, línea y columna; errores de caracteres/cadenas/números localizados.
RF-04	Parser real con precedencia y anidación; AST tipado y navegable con `span` de origen, exportable como JSON.
RF-05	Semántica: identificadores, tipos, inicialización, ámbitos, firmas de funciones, arrays, retorno, operadores y diagnósticos.
RF-06	Tabla de símbolos real por ámbito, tipos, posición de declaración, lecturas, escrituras y estado; resalta referencias de fuente.
RF-07	Diagnósticos estructurados con código, fase, severidad, span, explicación y solución; editor los marca y enfoca.
RF-08	Bloquea traducción y ejecución si fallan las validaciones; limpia artefactos anteriores o los etiqueta «desactualizados».
RF-09	Genera Python 3 y JavaScript de uso real, sin llamadas al backend para su lógica de negocio; runtime auxiliar incluido.
RF-10	`Leer`, `LeerTexto`, `Escribir`, expresiones y formatos equivalentes; entrada y salida por consola real, números y textos según reglas.
RF-11	`Si / Entonces / Sino / FinSi`, condiciones compuestas, comparaciones, `Y / O / No`, anidación y agrupación.
RF-12	Consola real y ejecutores aislados para Python y Node.js; stdin/stdout/stderr, códigos de salida, cancelación, límites y tiempo.
RF-13	Visualización trazable de léxico, sintaxis, semántica, generación y ejecución, basada en resultados técnicos calculados.
RF-14	Exportación de fuente, .py, .js, JSON del AST, tabla de símbolos, diagnóstico e informe de pruebas en JSON, Markdown y PDF; todas las descargas funcionan.
RF-15	`Para ... Hasta ... Hacer ... FinPara` con límites inclusivos, paso positivo/negativo, variables y acumuladores.
RF-16	`Dimension arreglo[n]`, lectura, escritura, acceso y asignación indexados con validación de rango e índice 1-based.
RF-17	Definir y llamar funciones, parámetros, argumentos, `Retornar`, `FinFuncion`, scopes y llamadas anidadas/ recursivas limitadas por sandbox.
RF-18	Biblioteca académica A–E, 3 casos oficiales por programa y pruebas inéditas; suite automática real en Python y JS.
RF-19	Proyectos locales: crear, renombrar, abrir, editar, guardar, duplicar, eliminar con confirmación y recuperación de sesión.

ID	Funcionalidad real y criterio de aceptación resumido
RF-20	Comparador entre fuentes y código generado, con resaltado y mapas línea/columna de pseudocódigo → destino.
RF-21	Ejecución por lotes suministrando secuencia de entradas y ejecución interactiva transmitiendo entrada mientras el proceso solicita `Leer`.
RF-22	Trazabilidad de ejecución: tiempo, hash de fuente, versión, destino, entradas de prueba protegidas, exit code y diagnóstico asociado.
RF-23	Matriz de pruebas editable desde UI: caso, entradas, salida esperada, resultado Python/JS, diff y descarga de reporte.
RF-24	Guardado automático controlado, aviso de cambios sin guardar y restauración local; no perder trabajo al recargar.
RF-25	Manual del lenguaje, gramática EBNF completa, ayuda contextual y ejemplos documentados cuya salida se comprueba.
RF-26	Modo defensa con recorrido guiado por fases y casos válidos/erróneos, invocando análisis y ejecución auténticos.
RF-27	API versionada y documentada (OpenAPI), errores HTTP coherentes, contratos tipados y trazas correlacionables.
RF-28	Extensiones integradas `Mientras`, booleanos, `Modulo`, cadenas y comentarios como se define en la gramática v2.
RF-29	Salud operativa de servicios y disponibilidad de runtimes/ sandbox; bloquear con explicación cuando la ejecución no está lista.

Cobertura: todos los RF son obligatorios en el resultado final. Su implementación puede ordenarse por dependencias, pero no declararse «fuera de alcance» ni «para después» para cerrar el proyecto.

4. Especificación formal del lenguaje fuente

La gramática es la fuente de verdad: el parser, los tests y la ayuda deben compartir sus reglas. No derivar comportamiento específico del nombre de archivo. Se mantienen las sentencias escritas por el docente. Las extensiones adicionales están explícitamente identificadas.

4.1 Convenciones léxicas y de formato

- Palabras clave insensibles a mayúsculas/minúsculas (`Inicio`, `INICIO`, `inicio`); identificadores **sensibles** a mayúsculas/minúsculas; no admitir nombres que colisionen con reservadas.
- Identificadores Unicode normalizados NFC, con letras al inicio, seguidas de letras, dígitos o `_`; limitar el conjunto aceptado de forma documentada y con diagnósticos exactos si difiere entre plataformas.
- Números enteros o decimales con punto; sin separadores de miles. `-` es operador unario. Literales booleanos `Verdadero` y `Falso` (extensión v2).
- Cadenas entre comillas dobles con escapes `\\`, `\"`, `\n`, `\t`; Unicode válido. Rechazar sin ambigüedad comillas no cerradas.

- Un encabezado, `Leer`, `Escribir`, asignación y `Retornar` terminan al final de línea; una sentencia por línea. Líneas vacías permitidas. Comentarios `//` a fin de línea (extensión v2). Saltos de línea dentro de cadenas solo con `\n`.
- Operadores: `+ - * / % = == != < <= > >= ( ) [ ] ,` ; `=` asignación, `==` igualdad, `%` módulo. Palabras `Y`, `O`, `No` para lógicos. Se admiten espacios e indentación opcionales; la estructura se determina con `Fin*`, no por sangría.
- La gramática también reconoce los **nombres de función con variable declarativa** del Programa E: `Funcion area = CalcularArea(base, altura)` ; `area` es el nombre de la variable de retorno declarada, `CalcularArea` el nombre invocable. También acepta `Funcion Nombre(params)`.

4.2 Gramática EBNF normativa

Notación: `{ X }` = repetición, `[ X ]` = opcional; `NL` = fin de línea; `IDENT`, `NUM`, `STRING` son terminales léxicos. `NL+` incluye uno o más saltos, ignorando líneas vacías o comentario puro cuando proceda. La expresión booleana se resuelve con parser de precedencia/Pratt, no con regex de bloques.

```
programa        = "Inicio", NL+, { elemento }, "Fin", [ NL+ ] ;
elemento        = sentencia | funcion ;
sentencia       = leer | leer_texto | escribir | asignacion | condicional
                | para | dimension | mientras | retornar | expr_stmt ;
leer            = "Leer", destino, NL+ ;
leer_texto      = "LeerTexto", destino, NL+ ;
escribir        = "Escribir", expresion, { ",", expresion }, NL+ ;
asignacion      = destino, "=", expresion, NL+ ;
destino         = IDENT, [ "[", expresion, "]" ] ;
dimension       = "Dimension", IDENT, "[", expresion, "]", NL+ ;
condicional     = "Si", expresion, "Entonces", NL+,
                  { sentencia }, [ "Sino", NL+, { sentencia } ], "FinSi", NL+ ;
para            = "Para", IDENT, "=", expresion, "Hasta", expresion,
                  [ "Paso", expresion ], "Hacer", NL+,
                  { sentencia }, "FinPara", NL+ ;
mientras        = "Mientras", expresion, "Hacer", NL+,
                  { sentencia }, "FinMientras", NL+ ;
funcion         = "Funcion", [ IDENT, "=" ], IDENT, "(", [ parametros ], ")", NL+,
                  { sentencia }, "FinFuncion", NL+ ;
parametros      = IDENT, { ",", IDENT } ;
retornar        = "Retornar", expresion, NL+ ;
expr_stmt       = llamada, NL+ ;
expresion       = disyuncion ;
disyuncion      = conjuncion, { "O", conjuncion } ;
conjuncion      = igualdad, { "Y", igualdad } ;
igualdad        = relacion, { ("==" | "!="), relacion } ;
relacion        = aditiva, { (">" | "<" | ">=" | "<="), aditiva } ;
aditiva         = multiplicativa, { ("+" | "-"), multiplicativa } ;
multiplicativa  = unaria, { ("*" | "/" | "%"), unaria } ;
unaria          = ("-" | "+" | "No"), unaria | primaria ;
primaria        = NUM | STRING | "Verdadero" | "Falso" | referencia
                | llamada | "(", expresion, ")" ;
referencia      = IDENT, [ "[", expresion, "]" ] ;
llamada         = IDENT, "(", [ argumentos ], ")" ;
argumentos      = expresion, { ",", expresion } ;
```

Nota para el agente: resolver la ambigüedad de `IDENT(...)` como llamada y `IDENT[...]` como indexación con lookahead; tratar encabezados de control como líneas completas. Aceptar `Fin` al final de archivo sin salto adicional. Implementar errores recuperables en parser para mostrar múltiples diagnósticos con seguridad y evitar bucles infinitos de recuperación. Mantener el archivo `docs/gramatica.ebnf` sincronizado con la gramática aplicada y sus pruebas de conformidad.

4.3 Semántica unificada

Aspecto	Política exigida
Números	Cálculos con semántica finita IEEE-754 de doble precisión en ambos destinos; división `/` real, resultado de `4/2` se muestra `2`; redondeo/formato por biblioteca compartida equivalente. Rechazar `NaN` / `Infinity` de entradas.
Enteros y precisión	Para indexación, tamaño y límites iterativos requerir enteros exactos y seguros (−(2^53−1) a 2^53−1); límites de recursos del sandbox controlan cálculos excesivos.
`Leer` / `LeerTexto`	`Leer` crea/inicializa variable o elemento a partir de entrada numérica finita, por compatibilidad A–E. `LeerTexto` lee entrada textual sin conversión numérica. Ninguna operación convierte números/cadenas silenciosamente; ambos destinos usan idéntica política.
`Escribir`	Concatena argumentos **sin separador implícito**; respeta espacios escritos dentro de literales, imprime un salto de línea al finalizar. El formato de números y booleanos se normaliza por helpers equivalentes en ambos destinos.
Ámbitos	Global de programa y local por función; las variables locales no se filtran a globales. No se permite leer símbolos antes de ser inicializados; considerar rutas de ejecución para asignaciones dentro de `Si`.
Condiciones	Tipo booleano explícito; comparaciones entre tipos compatibles. `Y` y `O` hacen cortocircuito, `No` invierte. `Si` no acepta números como verdad implícita.
`Para`	Límite inclusivo, paso por defecto `+1` si inicio ≤ fin, de otro modo `−1` como extensión definida; `Paso 0` error controlado. Inicio, límite y paso se evalúan una sola vez; variable de iteración entera.
Arreglos	`Dimension a[n]` crea n posiciones con índices **1..n**, como en Programa D; `n` debe ser entero positivo y seguro. Lectura/escritura fuera de rango es error de ejecución localizable; lectura de posición no inicializada es error de ejecución.
Funciones	Declaración de todas las firmas en una pasada previa, permite llamadas antes de la definición y recursión; parámetros por valor para primitivas y política explícita documentada para arreglos; `Retornar` solo dentro de función; rutas deben retornar valor en funciones usadas como expresión.
Errores	Variables desconocidas, duplicaciones de funciones, aridad, retornos inválidos y tipos incompatibles son semánticos. División por cero en expresión constante es diagnóstico preventivo; si depende de entrada es error en runtime, nunca caída abrupta.
Extensiones	`Mientras` evalúa expresión booleana en cada iteración; `%` numérico con prueba de paridad Python/JS incluso con operandos negativos, sin confiar en diferencias nativas.

Para reproducir exactamente las pruebas académicas el compilador debe respetar el algoritmo original, incluso cuando el usuario pudiera preferir una implementación distinta. No reescribir automáticamente operaciones de los ejemplos para forzar resultados.

4.4 Representación intermedia y contrato de diagnóstico

Cada nodo AST guarda `node_id`, `kind`, `span {start_line,start_column,end_line,end_column}`, hijos y anotaciones de tipo (estas últimas solo después del análisis semántico). La tabla de símbolos almacena `name`, `scope`, `kind`, `type`, `declaration_span`, `initialized`, `references`. Los datos serializados no contienen objetos ejecutables. Tokens y AST deben corresponder al texto de entrada y sus posiciones reales.

Diagnóstico canónico:
`{"code":"SEM_UNDEFINED_IDENTIFIER","stage":"semantic","severity":"error","message":"Variable 'x' no definida","span":{"start_line":3,"start_column":10,"end_line":3,"end_column":11},"hint":"Inicialícela antes de utilizarla"}`. Enumerar además códigos coherentes `LEX_INVALID_CHAR`, `LEX_UNTERMINATED_STRING`, `SYN_MISSING_ENDIF`, `SYN_UNEXPECTED_TOKEN`, `SEM_TYPE_MISMATCH`, `SEM_UNINITIALIZED`, `SEM_WRONG_ARITY`, `RUN_DIV_ZERO`, `RUN_BOUNDS`, `RUN_INPUT_MISSING`, `RUN_TIMEOUT`, `RUN_MEMORY`, `RUN_SANDBOX_UNAVAILABLE`, `RUN_CANCELLED`.

5. Arquitectura integral e interconexiones

```
NAVEGADOR (editor + inspector + proyectos + consola + pruebas)
     | HTTPS/HTTP loopback - REST JSON + WebSocket de ejecución
     v
API FastAPI (validación, rate limit, IDs de request, OpenAPI)
     |
     +--> PROJECT SERVICE -> almacenamiento local SQLite + archivos seguros
     |
     +--> COMPILER SERVICE
     |       source -> Lexer -> Parser/AST -> Semantic/SymbolTable
     |                                  |          |
     |                                  +--> Diagnostics/SourceMaps
     |                                             |
     |                                             +--> Python Codegen -> .py + helper
     |                                             +--> JS Codegen -> .js + helper
     |
     +--> RUN SERVICE -> sandbox supervisor -> Python 3 / Node.js
     |                                      -> stdin/stdout/stderr/exit/metrics
     |
     +--> TEST SERVICE -> fixtures A-E + nuevos casos + diff + reports
     v
FRONTEND: resultados consistentes asociados al hash de la misma fuente
```

Stack elegido: frontend TypeScript + Vite + React (editor Monaco o CodeMirror mantenido en el repositorio); API Python 3.12+ con FastAPI/Pydantic; lexer/parser/semántica/AST propios en Python, sin transpilación por reemplazo de cadenas; SQLite local para proyectos e historial, migraciones; generación Python 3 y JavaScript moderno para Node.js LTS instalado; ejecutor rootless OCI (Podman rootless preferido, Docker rootless aceptable) con imágenes aisladas para Python y Node; pytest y Hypothesis + Playwright + pruebas de integración end-to-end. Versiones exactas se fijan en lockfiles y documentación de compatibilidad.

El frontend **no** calcula tokens/AST ficticios ni determina por sí solo si la compilación es válida: consume la respuesta del servicio. El servicio jamás ejecuta código del usuario en el proceso FastAPI. El API guarda hashes para evitar ejecutar artefactos de una fuente distinta. El resultado de `compile` debe incluir `compiler_version`, `source_hash`, `artifact_hash` y `runtime_version`.

5.1 Esquema de archivos esperado

```
pseudo-studio/
  frontend/              # editor, paneles, API client, WS client, UI tests
  backend/
```

```
    app/                  # REST, validación, auth local, schemas
    compiler/             # tokens, lexer, parser, AST, semantic, symbols
    generators/           # python, javascript, shared conformance
    runtime/              # runner, broker de sesiones, sandbox, quotas
    projects/             # SQLite, migraciones, seguridad de archivos
    test_service/         # orquestación de suites, reportes y diferencias
    tests/                # unitarias, integración, property, seguridad
  docs/                   # gramatica.ebnf, ADR, manual, arquitectura, API
  examples/               # A-E exactos de la guía, entradas y expected
  sandbox/                # Containerfiles, runtime helpers, perfiles
  scripts/                # bootstrap, verify, smoke, cleanup, release
  .github/workflows/      # CI y escaneos
  README.md
  .env.example
  compose.yaml            # desarrollo local cuando corresponda
  pyproject.toml
  package.json            # si aplica al workspace raíz
```

5.2 Contratos API mínimos, operativos y versionados

Método / ruta	Petición mínima	Respuesta real
`GET /api/v1/health`	—	Estado de backend, base, compilador, sandbox y runtimes; sin secretos.
`POST /api/v1/analyze`	`source`	tokens, AST, símbolos, diagnósticos, fases, hash, estado.
`POST /api/v1/translate`	`source`, `targets: ["python","javascript"]`	dos artefactos genuinos, hashes, source maps; falla con 422 y diagnósticos.
`POST /api/v1/run`	`source`, `target`, `inputs.values[]`	`stdout`, `stderr`, `exit_code`, `duration_ms`, `run_id`, error normalizado; ejecuta en sandbox.
`WS /api/v1/run/interactive`	iniciar `source`, `target`; luego mensajes de entrada/cancelación	eventos `started`, `stdout`, `stderr`, `input_requested`, `exited`, `error`; eventos reales del proceso.
`GET /api/v1/examples`	—	lista A–E y metadatos; `GET /api/v1/examples/{id}` devuelve fuente y casos.
`POST /api/v1/tests/run`	`source` o `example_id`, `test_cases[]`	por caso Python/JS, esperado/real, diff, tiempos y totales.
`CRUD /api/v1/projects`	nombre, contenido, versión de edición	proyectos persistentes en SQLite; control de concurrencia y validación.
`GET /api/v1/exports/{id}`	id válido	descarga de archivo real autorizado, MIME correcto, sin traversal.

Versionar todas las solicitudes/respuestas con Pydantic y OpenAPI; usar `400` entrada inválida, `404` recurso ausente, `409` conflicto, `413` tamaño máximo, `422` errores de compilación, `429` control de peticiones, `503` sandbox inoperativo y `500` excepción interna sin filtración de información confidencial. `run` nunca devuelve «éxito» con exit code 0 si el proceso falló o fue cancelado. Definir tamaño máximo de fuente y cuotas razonables configurables que sean **protecciones operativas**, no límites arbitrarios a las características del lenguaje.

5.3 Ejecución segura y disponibilidad

Ejecutar los scripts generados como procesos reales exclusivamente dentro de contenedores OCI rootless efímeros; imágenes fijadas por digest; usuario no privilegiado; sistema de archivos raíz de solo lectura; directorio efímero controlado; `--network none`; capacidades Linux eliminadas; `no-new-privileges`; seccomp/aislamiento del host; límites de CPU, memoria, PIDs, archivo, tiempo de pared y bytes de stdout/stderr; limpieza al terminar y al cancelar. No montar home del usuario, secretos, el socket Docker/Podman ni rutas host sensibles; no exponer sandbox en internet. Evitar `shell=True`, `eval` y `exec` en API o en el propio servicio; usar comandos parametrizados y rutas creadas por servidor. El código generado **no** tiene privilegios del servidor web.

La disponibilidad del motor OCI y de imágenes runtime se comprueba antes de habilitar los botones Ejecutar; fallos se muestran como tales con instrucciones de reparación. Las pruebas deben comprobar aislamiento de red, escape de rutas, procesos colgados, output infinito, agotamiento de memoria y cancelación; no afirmar aislamiento correcto solo por usar contenedores. En sistemas que no permitan contenedores rootless, documentar instalación de VM aislada equivalente como dependencia operativa verificable; **no reemplazar ejecución por resultados prefijados**.

Para consola interactiva, definir un pequeño protocolo de stdin desde el runtime helper generado: un `Leer` emite la solicitud de input al broker; UI permite enviar valores conforme se piden, muestra los mensajes `Escribir` en el orden real y recibe fin/cancelación. La API de lote usa la **misma** ruta de ejecución con `inputs.values`, no un intérprete diferente. Limitar concurrencia y limpiar sesiones huérfanas.

6. Generación de código, equivalencias y exportación

El generador atraviesa AST validado, aplica indentación/bloques, emite source map e incluye/helpers de I/O y formato para uniformar comportamientos. Los scripts producidos deben poder ejecutarse **fuera de la interfaz** si se descargan junto a sus helpers o como artefacto autocontenido. Node.js usa entrada real (`readline` o protocol adapter de runtime); Python usa `sys.stdin` o adaptador equivalente. No generar código Python con JS embebido ni usar un solo intérprete para simular el otro.

| Construcción fuente | Python real | JavaScript real | |---|---|---| | `Escribir "Hola"` | `print` /helper de escritura | `process.stdout.write` /helper de escritura | | `Leer x` | lectura real de stdin + validación numérica | lectura real de stdin + validación numérica | | `x = (a+b)*c` | expresión AST con paréntesis correctos | expresión AST con paréntesis correctos | | `Si ... Entonces` | `if ...:` | `if (...) { ... }` | | `Y / O / No` | `and / or / not` | `&& / || / !` | | `Para i = 1 Hasta n Hacer` | iteración inclusiva mediante helper o rango ajustado | bucle con límites inclusivos | | `Dimension a[n]` | estructura con índices 1..n y controles | estructura con índices 1..n y controles | | `Funcion ... / Retornar` | `def` y `return` | `function` y `return` |

Una optimización opcional solo se acepta si no cambia los resultados ni la trazabilidad. `Escribir "Valor: ",x` debe emitir `Valor: 4`, no introducir separadores, ni cambiar `4` a `4.0` o `4.000000` sin justificación. Las funciones auxiliares de normalización de número, `%` y aritmética se prueban diferencialmente para ambos destinos.

7. Casos académicos obligatorios: A–E

Transcribir en `examples/` los cinco programas **respetando su sintaxis** como aparecen en la guía. Los siguientes resultados visibles son los de las tablas oficiales; además verificar los mensajes previos de solicitud de datos generados por `Escribir`. Crear **tres** casos en cada programa y ejecutarlos tanto en Python como en JS: **mínimo 30 ejecuciones reales** de referencia.

Programa	Datos de entrada	Resultado final esperado en consola
A, caso 1	`4,2,3`	`El resultado es: 16`
A, caso 2	`10,5,2`	`El resultado es: 28`
A, caso 3	`6,3,4`	`El resultado es: 34`
B, caso 1	`8,80`	`Aprobado`
B, caso 2	`7,60`	`Reprobado por asistencia`
B, caso 3	`4,90`	`Reprobado por nota`
C, caso 1	`4`	`El factorial es: 24` y `La suma es: 20`
C, caso 2	`5`	`El factorial es: 120 y La suma es: 30`
C, caso 3	`1`	`El factorial es: 1` y `La suma es: 2`
D, caso 1	`3;4,9,2`	`El promedio es: 5 y El número mayor es: 9`
D, caso 2	`4;10,20,5,15`	`El promedio es: 12.5` y `El número mayor es: 20`
D, caso 3	`5;1,1,1,1,1`	`El promedio es: 1 y El número mayor es: 1`
E, caso 1	`5,4`	`El área es: 20` y `Clasificación: Pequeña`
E, caso 2	`10,8`	`El área es: 80 y Clasificación: Grande`
E, caso 3	`6,9`	`El área es: 54` y `Clasificación: Grande`

Controles especiales: A evalúa precedencia y división; B condiciones anidadas y asistencia `>=75` tal como está escrita; C límite inclusivo y dos acumuladores; D dimensión y índices desde 1; E declaración/retorno y función a función. La rúbrica de la primera fase pide traducciones **manuales** completas de A/B por separado: el sistema no sustituye por sí mismo esa obligación documental. Si el docente añade un Programa F, incorporarlo como nuevo fixture basado en el enunciado recibido, no inventar un caso oficial.

8. Matriz transversal de verificación

8.1 Casos inválidos obligatorios

Caso	Fuente/entrada	Resultado verificable
N-01	`Escribir @`	error `LEX_INVALID_CHAR` con ubicación; 0 artefactos.

Caso	Fuente/entrada	Resultado verificable
N-02	`resultado = (a + b`	error sintáctico de paréntesis; 0 artefactos.
N-03	`Escribir total` antes de definir	error semántico de símbolo; 0 artefactos.
N-04	`Si x > 0 Entonces` sin `FinSi`	diagnóstico sintáctico de bloque sin cerrar.
N-05	repetir nombre de función o llamar con aridad errónea	error semántico con ubicación.
N-06	`Dimension a[3]`, acceder `a[0]` o `a[4]`	compilación válida; error controlado en runtime con source span.
N-07	A con `b=0`	error runtime de división entre cero en ambos destinos.
N-08	C/D con input no numérico o incompleto	error `RUN_INPUT_INVALID` / `RUN_INPUT_MISSING`; no stacktrace sin procesar al usuario.
N-09	`Paso 0` literal o calculado	error semántico preventivo si constante, runtime si dinámico.
N-10	bucle infinito en `Mientras`	timeout/cancelación real, proceso y contenedor terminados.
N-11	instrucciones que intentan red, rutas host o invocación de shell vía texto de entrada	fuente fuera de gramática rechazada o aislamiento bloquea acción; no acceso indebido.
N-12	dos ejecuciones simultáneas, una cancelada	aislamiento por sesión, recursos liberados, otra ejecución sin daño.

8.2 Pruebas inéditas de equivalencia

Crear por lo menos tres programas válidos inéditos y tres inválidos inéditos adicionales a A–E, sin codificar excepciones específicas para ellos. Incluir condiciones `Y/O/No`, `Mientras`, recursión, manejo de números negativos con `%`, arreglos y precedencia con paréntesis. Para cada válido: stdout exacto en Python y Node.js con inputs definidos y prueba de comparación normalizada, conservando por separado la salida cruda. Probar límites: `n=1`, `n=0` donde aplique, `nota=6`, `asistencia=75`, números decimales, strings con escapes, espacios y saltos de línea, Unicode y datos mal formados.

8.3 Criterios medibles de aceptación

- A–E: **15/15 casos** pasan en Python y **15/15 casos** pasan en Node.js con los resultados indicados y sin fallas ocultas.
- Suite de inéditos y negativos: **100 % de los casos obligatorios** pasan, con diagnósticos y ubicación cuando proceda.
- Cada RF-01..RF-29 dispone de una prueba verificable o evidencia técnica reproducible; ninguna funcionalidad simulada.
- Los generadores producen scripts invocables por sus intérpretes reales y descargables en disco; ningún script depende de ejecución del backend para la lógica del programa.
- Fuente editada invalida artefactos de compilaciones anteriores; el UI nunca ejecuta código viejo como nuevo.

- Pruebas unitarias, integraciones API, equivalencia de destinos, e2e navegador, validaciones de seguridad y smoke de instalación pasan en CI y en máquina local documentada.
- Instalación desde clon limpio con un comando de bootstrap documentado; salud verde después de levantar backend, frontend y sandbox; `README.md` y manual permiten reproducir resultados.
- WCAG 2.2 AA como objetivo verificable de accesibilidad para flujos principales: teclado, focus visible, ARIA, contraste; automatización y revisión manual.
- Rendimiento objetivo medido y documentado, por ejemplo análisis de 1.000 líneas ≤2s en máquina de referencia; no falsear pruebas en máquinas de hardware desconocido.

9. Requisitos no funcionales y operación (RNF)

ID	Condición exigida
RNF-01	Determinismo de generación, versiones fijadas y hashes verificables de código.
RNF-02	APIs con esquema, sanitización de salida, límites configurables y tratamiento de excepciones.
RNF-03	Seguridad defensiva: rootless sandbox, sin red, filesystem efímero, no secretos, cuotas, cancelación, pruebas de evasión.
RNF-04	Privacidad local por defecto: no envío de código ni entradas a terceros; telemetría deshabilitada salvo consentimiento explícito.
RNF-05	Disponibilidad y diagnóstico: healthcheck de API, motor y contenedores; errores accionables sin fingir disponibilidad.
RNF-06	Accesibilidad y responsividad en escritorio y móvil con navegación por teclado.
RNF-07	Observabilidad técnica con logs estructurados y correlation ID; nunca exponer entradas sensibles en logs.
RNF-08	Persistencia fiable de proyectos, migraciones, backup/ exportación, integridad y restauración.
RNF-09	Calidad del código: tipado, lint/format, tests, dependencias bloqueadas y CI de release.
RNF-10	Interoperabilidad real entre Windows/macOS/Linux mediante instrucciones y script compatibles donde sea posible.
RNF-11	Reproducibilidad total: fixtures, resultados de ejecución, hashes, fecha, runtime, manifiestos y reportes.
RNF-12	Rendimiento sostenible con límites de recursos **de seguridad**, sin falsear capacidades ni suprimir construcciones admitidas.

10. Diseño de persistencia, estado y sincronización

Persistir proyectos y revisiones en SQLite local con campos `id`, `name`, `source`, `source_hash`, `created_at`, `updated_at`, `version` y metadatos de compilación opcionales. Las entradas de usuario usadas en una prueba no se guardan por defecto; al guardar un caso de prueba el usuario debe consentirlo. Aplicar migraciones versionadas y copias de seguridad local documentadas. El frontend conserva borrador sin perder datos al cambiar pestañas, pero al editar la fuente invalida el hash asociado a análisis/código/ejecución. El UI debe mostrar «código desactualizado» y exigir recomputar. Las listas académicas se distribuyen como fixtures de solo lectura; el usuario puede duplicarlas como proyectos editables.

11. Observabilidad funcional y modo defensa

El inspector muestra conteo y secuencia de tokens reales (con posición), AST navegable por node_id, lista de símbolos por ámbito, diagnósticos y mapas de origen. Seleccionar un token/nodo/diagnóstico resalta el segmento en el editor; seleccionar línea de destino muestra su procedencia; la vista de fases indica tiempos medidos y estado obtenido realmente de los servicios. En modo defensa, el botón «Siguiente fase» dispara o presenta resultados calculados del pipeline sobre el texto actual y permite alternar entre caso correcto y incorrecto, con logs/exportables. Está prohibido dibujar «22 tokens», «AST válido» o «salida 18» cuando esas cifras no proceden de un cálculo.

12. Entregables físicos exigidos al agente constructor

1. Repositorio con frontend, API, compilador, ambos generadores, sandbox, persistencia, fixtures A–E y tests reales.
2. `docs/gramatica.ebnf` normativo completo, especificación de semántica y decisiones de arquitectura (ADR).
3. Scripts de bootstrap, start/stop y verificación; Docker/Podman rootless definido y comprobado.
4. `README.md` con requisitos, instalación limpia, comandos, URLs, prueba de salud, ejecución y solución de problemas.
5. Suite CI con resultados, reportes por requisito y artefactos descargables; fixture oficial × destino × caso.
6. Manual de usuario, guía de defensa, screenshots y/o grabación **obtenidos desde la aplicación real**, sin representar renders como funcionalidad probada.
7. Informe final de trazabilidad RF-01..29 y RNF-01..12 con estado, evidencia, tests, hash/commit y pendientes reales.
8. Código generado para programas A–E en los dos destinos y ejecución de las 30 pruebas documentadas.
9. Archivo de decisiones/riesgos: dudas académicas resueltas cuando haya evidencia y obstáculos que requieran permisos o dependencias; nunca ocultar bloqueos.

13. Definición de terminado y protocolo de verificación

Una tarea individual solo se marca `[x]` cuando la funcionalidad está implementada, es invocable por otra capa, pasa los tests definidos, tiene manejo de errores, y se adjunta ruta/artefacto/evidencia. Un componente con pruebas unitarias pero sin conexión al UI **no está terminado**; un UI con respuestas fijas **no está terminado**; un sandbox sin haberlo ejecutado **no está terminado**. Los requisitos solo se consideran cerrados cuando se supera la matriz de pruebas y los controles de seguridad, incluyendo arranque limpio y revisión de las interfaces en navegador real.

La totalidad del producto está terminada cuando cada RF y RNF está trazado y validado, los 30 casos de A–E y los casos inéditos pasan en ambos destinos, el runner aísla procesos, la consola interactiva funciona, no existen controles inertes y el repositorio puede reinstalarse usando la documentación.

14. Incertidumbres preservadas de los documentos académicos

- `fase_1.pdf` declara 22 % en la primera página, 20 % en la segunda y la rúbrica: no presentar uno como confirmado sin aclaración formal.
- La guía indica 11 de octubre para entrega/defensa; verificar hora y modalidad en el canal académico oficial.
- En el documento fuente recibido hay programas A–E, no se ha aportado un F. Un F no debe inventarse como parte del anexo.
- La comparación de asistencia de B es `>=75` en el programa escrito; esta especificación y sus pruebas respetan ese borde.
- El material original no formaliza completamente convenciones de tipos/índices y semántica; las secciones 4.1–4.3 de esta v2.0 **toman decisiones de ingeniería explícitas** que deberán implementarse y documentarse, conservando la compatibilidad A–E.
- La entrega documental de fase 1 (portada, índice, objetivos, introducción, arquitectura, EBNF, dos lenguajes, traducciones manuales A/B, conclusiones, bibliografía APA 7) es independiente del plan técnico y no debe eliminarse.

15. Correspondencia con documentos originales y control de versión

- **v1.0 → v2.0:** se conservan los RF-01..RF-18 pero desaparecen las etiquetas MVP/V2; todos los módulos son obligatorios; se añade RF-19..29, especificación de ejecución interactiva, persistencia, pruebas completas A–E, aislamiento, integraciones, equivalencia diferencial y evidencia de aceptación.
- **Fuente académica:** `fase_1.pdf`, págs. 1–2 (fase documental y rúbrica), págs. 3–5 (casos A–E), pág. 6 (criterios).
- **Fundamentos:** `UIA2.pdf`, unidad 2 (autómatas finitos y transiciones); `UIA3.pdf`, unidad 3 (lenguajes regulares, expresiones regulares, gramáticas y límites de analizadores regulares).
- **Control de cambios:** versión 2.0 / 10-10-2026 / ampliación técnica integral solicitada por el dueño del proyecto. La implementación, las pruebas y el estado de ejecución todavía deben producirse por el agente constructor.